

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – PSEUDOCODE WRITING TASK

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO5 – Naturalization (BL4, BL6)	Assessment:	Assessment Tool 1 – Pseudocode Writing Task
Weightage:	35% of CIA	Total Marks:	100
CO5 Statement:	<i>Construct MST using shortest path algorithms and traverse the graph to model and analyse realworld problem.</i>		
Student Name:	Sasikumar Megha	Reg. No.:	192471038
Section / Batch:	3 rd Cs&Bs	Date:	31/08/2026

Code of Conduct

I _____ Sasikumar Megha _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the Student: _____
Sasikumar Megha _____

Instructions

- This test contains 5 Pseudocode Writing Task questions. Total: 100 Marks.
- When a student answer using pseudocode writing task should evaluate based on the ability to design clear, logical, and efficient pseudocode solutions for data structure problems.
- No negative marking. Unanswered questions carry zero marks.

Section / Topic	Focus	Q Nos	Marks
Minimum Spanning Tree	Kruskal's Algorithm	1	20
Graph Sorting	Topological Sort	2	20

Shortest Path Algorithm	MST & Dijkstra's Algorithm	3	20
Shortest Path Algorithm	Dijkstra's Algorithm	4	20
Minimum Spanning Tree	Prim's or Kruskal's Algorithm	5	20
GRAND TOTAL:		100 Marks	

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20	Demonstrates complete and clear understanding; handles edge cases effectively	Good understanding; minor gaps in edge case handling	Partial understanding; misses some requirements	Misinterprets problem; major gaps
Code Correctness	30	Fully correct; passes all test cases	Mostly correct; minor errors	Partially correct; several test cases fail	Incorrect or nonfunctional code
Code Quality & Readability	20	Clean, modular, wellcommented, follows standards	Readable with some structure	Limited readability; poor structure	Unorganized and hard to read
Complexity Analysis	20	Accurate time & space complexity with justification	Correct but limited explanation	Incomplete or partially correct	Missing or incorrect
Testing & Validation	10	Thorough testing including edge cases	Adequate testing with few edge cases	Minimal testing	No testing evidence
Faculty In-charge		Course Coordinator		Program Director	
Signature: _____		Signature: _____		Signature: _____	
Date: _____		Date: _____		Date: _____	

PSEUDOCODE WRITING TASK

[← Back](#)

QUESTIONS

Q1 ✓
Minimum Spanning Tree -
Kruskal's Algorithm
20 marks

Q2 ✓
Graph - Topological Sort
20 marks

Q3 ✓
Shortest Path Algorithm -
Minimum Spanning Tree &
Dijkstra's
20 marks

Q4 ✓
Shortest Path Algorithm -
Dijkstra's Algorithm
20 marks

Q5 ✓
Minimum Spanning Tree -
Prim's or Kruskal's Algorithm
20 marks

5/5 answered

Q1 Minimum Spanning Tree - Kruskal's Algorithm

20 marks

Write pseudocode for constructing an MST using Kruskal's Algorithm. A city planning department needs to connect multiple zones with underground fiber-optic cables. Each zone is represented as a vertex, and possible cable connections between zones are represented as edges with installation costs (weights).

Constraints:

- All zones must be connected
- Total installation cost must be minimized
- No redundant connections (no cycles) allowed
- The network must remain fully connected

Your Answer

```
KruskalMST(G):  
  MST = empty set  
  for each vertex v in G:  
    MakeSet(v)  
  
  sort all edges of G in non-decreasing order of weight  
  
  for each edge (u, v) in sorted edges:  
    if FindSet(u) != FindSet(v): // no cycle will be formed  
      add (u, v) to MST  
      Union(u, v)  
  
  if size of MST = number of vertices - 1:  
    break  
  
  return MST
```

[Save Answer](#)

[Next →](#)

PSEUDOCODE WRITING TASK

← Back

QUESTIONS

Q1
Minimum Spanning Tree -
Kruskal's Algorithm
20 marks**Q2**
Graph - Topological Sort
20 marks**Q3**
Shortest Path Algorithms -
Minimum Spanning Tree &
Dijkstra's Algorithm
20 marks**Q4**
Shortest Path Algorithms -
Dijkstra's Algorithm
20 marks**Q5**
Minimum Spanning Tree -
Prim's or Kruskal's Algorithm
20 marks

3/5 answered

Q2 Graph - Topological Sort

20 marks

A DAG represents task dependencies. Write pseudocode for Topological Sort using DFS with cycle detection.

Constraints:

- a. Detect cycle before sorting
- b. Use DFS approach

Your Answer

```
TopologicalSort(int V, int adj[][MAX], int visited[], int recStack[], int stack[], int *top)
{
    for (int i = 0; i < V; i++)
        visited[i] = 0, recStack[i] = 0;

    for (int i = 0; i < V; i++)
    {
        if (visited[i] == 0)
        {
            if (DFS(i, adj, visited, recStack, stack, top) == 0)
            {
                printf("Cycle detected. Topological sort not possible");
                return;
            }
        }
    }

    while (*top != -1)
        printf("%d ", stack[*top--]);
}

int DFS(int u, int adj[][MAX], int visited[], int recStack[], int stack[], int *top)
{
    visited[u] = 1;
    recStack[u] = 1;

    for (int v = 0; v < V; v++)
    {
        if (adj[u][v] == 1)
        {
            if (visited[v] == 0)
            {
                if (DFS(v, adj, visited, recStack, stack, top) == 0)
                    return 0;
            }
            else if (recStack[v] == 1)
            {
                return 0; // cycle found
            }
        }
    }

    recStack[u] = 0;
    stack[++(*top)] = u;
    return 1;
}
```

Save Answer

Next →

QUESTIONS

Q1

Minimum Spanning Tree -
Kruskal's Algorithm
20 marks

Q2

Graph - Topological Sort
20 marks

Q3

Shortest Path Algorithms -
Minimum Spanning Tree &
Dijkstra's
20 marks

Q4

Shortest Path Algorithms -
Dijkstra's Algorithm
20 marks

Q5

Minimum Spanning Tree -
Prim's or Kruskal's Algorithm
20 marks

5/5 answered

Q3 Shortest Path Algorithm - Minimum Spanning Tree & Dijkstra's

20 marks

Write pseudocode integrating MST + Dijkstra for Smart transportation system.

Constraints:

- a. Use MST for infrastructure
- b. Use shortest path for routing

Your Answer

```
SmartTransportationSystem(int n, int graph[][MAX], int source)
{
    int mst[MAX][MAX];
    int parent[MAX], key[MAX], visited[MAX];
    int dist[MAX], sptSet[MAX];

    // Step 1: Build infrastructure using MST (Prim's Algorithm)
    for (int i = 0; i < n; i++)
    {
        key[i] = INF;
        visited[i] = 0;
        parent[i] = -1;
    }

    key[0] = 0;

    for (int count = 0; count < n - 1; count++)
    {
        int u = MinKeyVertex(key, visited, n);
        visited[u] = 1;

        for (int v = 0; v < n; v++)
        {
            if (graph[u][v] && visited[v] == 0 && graph[u][v] < key[v])
            {
                parent[v] = u;
                key[v] = graph[u][v];
            }
        }
    }

    // Store MST edges in infrastructure network
    for (int i = 1; i < n; i++)
    {
        mst[parent[i]][i] = graph[parent[i]][i];
        mst[i][parent[i]] = graph[i][parent[i]];
    }

    // Step 2: Find shortest route using Dijkstra's Algorithm
    for (int i = 0; i < n; i++)
    {
        dist[i] = INF;
        sptSet[i] = 0;
    }

    dist[source] = 0;

    for (int count = 0; count < n - 1; count++)
    {
        int u = MinDistance(dist, sptSet, n);
        sptSet[u] = 1;

        for (int v = 0; v < n; v++)
        {
            if (!sptSet[v] && mst[u][v] && dist[u] != INF &&
                dist[u] + mst[u][v] < dist[v])
            {
                dist[v] = dist[u] + mst[u][v];
            }
        }
    }

    // Output shortest route distances
    PrintDistances(dist, n);
}
```

[Save Answer](#)[Next →](#)

QUESTIONS

Q1 Minimum Spanning Tree - Kruskal's Algorithm
20 marks

Q2 Graph - Topological Sort
20 marks

Q3 Shortest Path Algorithms - Minimum Spanning Tree & Dijkstra's
20 marks

Q4 Shortest Path Algorithms - Dijkstra's Algorithm
20 marks

Q5 Minimum Spanning Tree - Prim's or Kruskal's Algorithm
20 marks

5/5 answered

Q4 Shortest Path Algorithm - Dijkstra's Algorithm

20 marks

Shortest Path with "Toll-free" Voucher: You have a voucher that allows you to traverse one edge for free (weight = 0). Write pseudocode to find the shortest path from S to D utilizing this voucher optimally.

Your Answer

```

ShortestPathWithVoucher(int n, int graph[][MAX], int S, int D)
{
    int dist[MAX][2];
    int visited[MAX][2];
    int i, j;

    for (i = 0; i < n; i++)
    {
        dist[i][0] = INF; // voucher not used
        dist[i][1] = INF; // voucher used
        visited[i][0] = 0;
        visited[i][1] = 0;
    }

    dist[S][0] = 0;

    for (int count = 0; count < 2 * n; count++)
    {
        int u = -1, used = -1, min = INF;

        for (i = 0; i < n; i++)
        {
            for (j = 0; j < 2; j++)
            {
                if (visited[i][j] == 0 && dist[i][j] < min)
                {
                    min = dist[i][j];
                    u = i;
                    used = j;
                }
            }
        }

        if (u == -1)
            break;

        visited[u][used] = 1;

        for (int v = 0; v < n; v++)
        {
            if (graph[u][v] != 0)
            {
                // Normal move
                if (dist[u][used] + graph[u][v] < dist[v][used])
                    dist[v][used] = dist[u][used] + graph[u][v];

                // Use voucher on this edge, if not used before
                if (used == 0 && dist[u][0] < dist[v][1])
                    dist[v][1] = dist[u][0];
            }
        }
    }

    if (dist[D][0] < dist[D][1])
        printf("Shortest distance = %d", dist[D][0]);
    else
        printf("Shortest distance = %d", dist[D][1]);
}

```

Save Answer

Next →

QUESTIONS

Q1

Minimum Spanning Tree -
Kruskal's Algorithm
20 marks

Q2

Graph - Topological Sort
20 marks

Q3

Shortest Path Algorithm -
Minimum Spanning Tree &
Dijkstra's
20 marks

Q4

Shortest Path Algorithm -
Dijkstra's Algorithm
20 marks

Q5

Minimum Spanning Tree -
Prim's or Kruskal's Algorithm
20 marks

5/5 answered

Q5 Minimum Spanning Tree - Prim's or Kruskal's Algorithm

20 marks

Second Best Spanning Tree: Design pseudocode to find the "Second Best" MST.

Hint: Find the MST first, then for every edge in the MST, try removing it and finding the next best edge to replace it.

Your Answer

```

SecondBestMST(int V, int E, Edge edges[])
{
    Edge mst[V - 1];
    int mstCost = KruskalMST(V, E, edges, mst);
    int secondBest = INF;

    for (int i = 0; i < V - 1; i++)
    {
        Edge removed = mst[i];
        int cost = 0;
        int count = 0;
        Edge tempTree[V - 1];

        // add all MST edges except the removed one
        for (int j = 0; j < V - 1; j++)
        {
            if (j != i)
            {
                tempTree[count++] = mst[j];
                cost = cost + mst[j].weight;
            }
        }

        // find the cheapest edge that reconnects the tree
        SortEdgesByWeight(edges, E);

        for (int k = 0; k < E; k++)
        {
            if (edges[k] is not removed edge)
            {
                if (FindSet(edges[k].u) != FindSet(edges[k].v))
                {
                    tempTree[count++] = edges[k];
                    cost = cost + edges[k].weight;
                    break;
                }
            }
        }

        if (count == V - 1 && cost > mstCost && cost < secondBest)
            secondBest = cost;
    }

    printf("Second Best MST Cost = %d", secondBest);
}

```

Save Answer